

Curso

Desarrollo Full Stack con C# y Angular

Docente: Edwin Barrientos



Sesión 5 — Módulo 2:

Repositorios, Servicios y CRUD



Objetivo del módulo: implementar la capa de aplicación y acceso a datos mediante los patrones Repository y Unit of Work, construyendo endpoints REST con paginación y filtros.

Agenda

- Patrón Repository: Desacoplando la lógica de negocio de EF Core
- Patrón Unit of Work: Consistencia transaccional entre repositorios
- Servicios de Aplicación y DTOs: Contratos públicos seguros
- Endpoints CRUD completos en ASP.NET Core Web API
- Paginación, filtrado dinámico y ordenamiento profesional

Sesión 1:

Repositorios, Servicios y CRUD



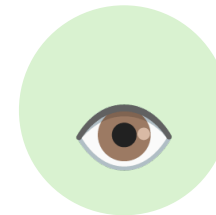
¿Por qué desacoplar el acceso a datos?

Clean Architecture en acción: la lógica de negocio no debe casarse con la tecnología de base de datos. Si mañana cambiamos de ORM o de motor, el núcleo de la empresa no se entera.



Testabilidad total

Pruebas unitarias de servicios usando mocks en memoria sin levantar bases de datos.



Consultas limpias

Centraliza la lógica de filtrado y ordenamiento en un único lugar reutilizable.



Límites claros

El controlador atiende HTTP, el servicio orquesta y el repositorio persiste.

El patrón Repository

Una colección en memoria simulada: el repositorio expone métodos de colección que ocultan las sentencias SQL y la complejidad de EF Core.

IRepository<T> Contrato genérico con operaciones comunes: ObtenerPorId, Listar, Agregar, Eliminar.

Repositorios específicos ISolicitudRepository agrega métodos del dominio: ListarPorEstado, ObtenerConDetalles.

Inversión de dependencias Application define la interfaz y Infrastructure entrega la implementación concreta.

Sin SaveChanges directo El repositorio no decide cuándo hacer commit: esa responsabilidad es del Unit of Work.

- Evita duplicar consultas complejas de LINQ a lo largo de varios controladores.

El contrato IRepository genérico

IRepository.cs

```
public interface IRepository<T> where T : class
{
    Task<T?> ObtenerPorIdAsync(int id);
    Task<IEnumerable<T>> ListarTodosAsync( );
    Task AgregarAsync(T entidad);
    void Actualizar(T entidad);
    void Eliminar(T entidad);
}
// Actualizar y Eliminar son síncronos: marcan el estado en el ChangeTracker
```

Repositorio especializado del dominio

ISolicitudRepository.cs

```
public interface ISolicitudRepository : IRepository<Solicitud>
{
    Task<Solicitud?> ObtenerConDetallesAsync(int id);
    Task<IReadOnlyList<Solicitud>> ListarPorSolicitanteAsync(int usuarioId);
    Task<IReadOnlyList<Solicitud>> ListarPorEstadoAsync(EstadoSolicitud estado);
    Task<(IReadOnlyList<Solicitud> Items, int Total)>
        ListarPaginadoAsync(int pagina, int tamano, EstadoSolicitud? estado);
}
```

Implementación en Infrastructure

SolicitudRepository.cs

```
public class SolicitudRepository : ISolicitudRepository
{
    private readonly SolicitudesDbContext _db;
    public SolicitudRepository(SolicitudesDbContext db) => _db = db;

    public async Task<Solicitud?> ObtenerConDetallesAsync(int id) =>
        await _db.Solicitudes
            .Include(s => s.Solicitante)
            .Include(s => s.Detalles)
            .Include(s => s.Aprobaciones)
            .FirstOrDefaultAsync(s => s.Id == id);
}
```

El patrón Unit of Work

Un solo árbitro para confirmar cambios: mantiene una lista de entidades afectadas y coordina el guardado de múltiples repositorios en una sola transacción.



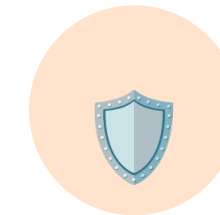
Consistencia

O se guardan la Solicitud y sus 5 Detalles, o nada se guarda.



Una sola llamada

Evita múltiples llamadas a la base de datos dentro del mismo caso de uso.



Aislamiento

Permite hacer Rollback ante cualquier fallo inesperado.

Interfaz e implementación de Unit of Work

UnitOfWork.cs

```
public interface IUnitOfWork : IDisposable
{
    ISolicitudRepository Solicitudes { get; }
    IUsuarioRepository Usuarios { get; }
    Task<int> CompletarAsync( );
}

public class UnitOfWork : IUnitOfWork
{
    private readonly SolicitudesDbContext _db;
    public UnitOfWork(SolicitudesDbContext db) => _db = db;
    public async Task<int> CompletarAsync( ) => await _db.SaveChangesAsync( );
}
```

Ciclos de vida en Dependency Injection

La regla de oro en Web APIs: elegir mal el ciclo de vida provoca fugas de memoria o errores de concurrencia fatales.



Transient

Nueva instancia cada vez que se inyecta. Ideal para cálculos livianos sin estado.



Scoped (El estándar)

Una instancia única por cada petición HTTP. Es el ciclo obligatorio para DbContext y UnitOfWork.



Singleton

Una sola instancia para toda la aplicación viva. Para cachés o clientes HTTP reutilizables.

Servicios de Aplicación: Orquestación

SolicitudService.cs

```
public class SolicitudService : ISolicitudService
{
    private readonly IUnitOfWork _uow;
    public SolicitudService(IUnitOfWork uow) => _uow = uow;

    public async Task<SolicitudDto> CrearAsync(CrearSolicitudDto dto, int usuarioId)
    {
        var solicitud = new Solicitud(dto.Motivo, usuarioId, dto.CentroCostoId);
        foreach (var item in dto.Detalles) solicitud.AgregarDetalle(item.Concepto, item.Importe);
        await _uow.Solicitudes.AgregarAsync(solicitud);
        await _uow.CompletarAsync( );
        return SolicitudMapper.ToDto(solicitud);
    }
}
```

DTOs: El contrato con el cliente

SolicitudDtos.cs

```
// DTOs inmutables con records: limpios y seguros
public record CrearSolicitudDto(
    string Motivo, int CentroCostoId, List<CrearDetalleDto> Detalles);

public record CrearDetalleDto(string Concepto, decimal Importe);

public record SolicitudDto(
    int Id, string Motivo, decimal MontoTotal, string Estado,
    string SolicitanteNombre, DateTime FechaCreacion,
    IReadOnlyList<DetalleDto> Detalles);
// El cliente jamás ve claves internas innecesarias ni campos sensibles
```

¿Por qué prohibido devolver Entidades?



Devolver Entidades

- ✗ Expone campos confidenciales como PasswordHash
- ✗ Errores de serialización JSON por referencias circulares
- ✗ Rompe el frontend al cambiar cualquier columna en BD
- ✗ Permite Over-Posting: el cliente modifica datos no autorizados



Devolver DTOs

- ✓ Contrato estricto y versionable diseñado para la vista
- ✓ Objetos planos sin referencias cíclicas
- ✓ Desacopla la evolución de la BD respecto de la Web
- ✓ Seguridad total contra asignaciones masivas maliciosas

El Controlador: Endpoints de Consulta

SolicitudesController.cs

```
[ApiController]
[Route("api/[controller]")]
public class SolicitudesController : ControllerBase
{
    private readonly ISolicitudService _srv;
    public SolicitudesController(ISolicitudService srv) => _srv = srv;

    [HttpGet("{id:int}")]
    public async Task<ActionResult<SolicitudDto>> Obtener(int id)
    {
        var resultado = await _srv.ObtenerPorIdAsync(id);
        return resultado is null ? NotFound( ) : Ok(resultado);
    }
}
```

El Controlador: Crear y Devolver 201

CrearEndpoint.cs

```
[HttpPost]
public async Task<ActionResult<SolicitudDto>> Crear(CrearSolicitudDto dto)
{
    int usuarioActualId = 1; // En S13 vendrá del Token JWT
    var creada = await _srv.CrearAsync(dto, usuarioActualId);
    return CreatedAtAction(
        nameof(Obtener), new { id = creada.Id }, creada);
}
// CreatedAtAction incluye la cabecera HTTP Location con la URL del nuevo recurso
```

Paginación profesional en la API

PagedResult.cs

```
// Envoltorio estándar para cualquier lista paginada
public record PagedResult<T>(
    IReadOnlyList<T> Items, int TotalRegistros, int Pagina, int TamanoPagina)
{
    public int TotalPaginas => (int)Math.Ceiling(TotalRegistros / (double)TamanoPagina);
    public bool TienePaginaAnterior => Pagina > 1;
    public bool TienePaginaSiguiente => Pagina < TotalPaginas;
}
```

Paginación en base de datos: Skip y Take

SkipTake.cs

```
// El cálculo se hace en SQL Server / SQLite con OFFSET y FETCH
public async Task<PagedResult<SolicitudDto>> ListarPaginadoAsync(
    int pagina, int tamano)
{
    var total = await _db.Solicitudes.CountAsync( );
    var items = await _db.Solicitudes
        .OrderByDescending(s => s.FechaCreacion)
        .Skip((pagina - 1) * tamano)
        .Take(tamano)
        .Select(s => SolicitudMapper.ToDto(s))
        .ToListAsync( );
    return new PagedResult<SolicitudDto>(items, total, pagina, tamano);
}
```

Filtrado dinámico en LINQ

FiltradoDinamico.cs

```
// Los filtros se acumulan antes de ejecutar la consulta
IQueryable<Solicitud> query = _db.Solicitudes.AsNoTracking( );

if (filtro.Estado.HasValue)
if (!string.IsNullOrEmpty(filtro.TextoBuscar))
    query = query.Where(s => s.Motivo.Contains(filtro.TextoBuscar));
if (filtro.MontoMinimo.HasValue)

// La consulta SQL final contiene solo los WHERE que se activaron
```

Pruebas unitarias de servicios con Moq

ServiceTests.cs

```
// Probamos SolicitudService simulando IUnitOfWork
var mockRepo = new Mock<ISolicitudRepository>( );
var mockUow = new Mock<IUnitOfWork>( );
mockUow.Setup(u => u.Solicitudes).Returns(mockRepo.Object);

var servicio = new SolicitudService(mockUow.Object);
var resultado = await servicio.CrearAsync(dto, 1);

Assert.NotNull(resultado);
mockUow.Verify(u => u.CompletarAsync( ), Times.Once);
```

Checklist de calidad para APIs CRUD

Estándares profesionales para la industria: una API bien construida es predecible, segura y fácil de consumir por Angular.

- ✓ Endpoints de lectura siempre paginados: jamás devuelvas un SELECT sin límite.
- ✓ Usa DTOs específicos de entrada y salida desacoplados de la base de datos.
- ✓ Devuelve códigos HTTP semánticos (200, 201 Created con cabecera Location, 404).
- ✓ Mantén los controladores delgados (Thin Controllers) delegando la lógica al servicio.
- ✓ Registra dependencias con Scoped para evitar mezclar estados entre peticiones.

Errores comunes en CRUD y repositorios



Evita esto

- ✗ Inyectar DbContext directamente en el Controlador
- ✗ Llamar SaveChanges dentro de un bucle foreach
- ✗ Devolver entidades con todas sus propiedades al cliente
- ✗ Hacer paginación en memoria después de ToList()
- ✗ Retornar siempre HTTP 200 aunque ocurra un fallo



Haz esto

- ✓ Inyectar servicios de aplicación en el Controlador
- ✓ Acumular cambios y llamar CompletarAsync una sola vez
- ✓ Mapear a DTOs con las propiedades estrictamente requeridas
- ✓ Ejecutar Skip() y Take() en SQL antes de ToListAsync()
- ✓ Usar códigos de estado REST exactos y estandarizados

Resumen de la sesión



Repository

Acceso a datos desacoplado y testeable.



Unit of Work

Consistencia transaccional en un solo commit.



DTOs

Contratos limpios sin exponer el dominio.



Controladores

Thin Controllers orquestando respuestas REST.



Paginación

Skip y Take optimizados desde base de datos.



Pruebas

Servicios testeables al 100% con Moq.

PREGUNTAS...



1. ¿Por qué usar Repositorios si EF Core ya implementa DbContext y DbSet?
2. ¿Qué ventaja nos da el Unit of Work frente a llamar SaveChanges en cada repositorio?
3. ¿Por qué nunca debemos devolver una entidad del dominio directamente al frontend?
4. ¿Cómo implementamos paginación eficiente sin traer miles de filas a memoria?
5. ¿Qué ciclo de vida de DI (Scoped, Transient, Singleton) debe tener cada componente?

Conclusiones

- ✓ El patrón Repository centraliza las consultas y permite probar la aplicación sin tocar la base de datos.
- ✓ Unit of Work asegura que múltiples operaciones se guarden o fallen de manera atómica.
- ✓ Los DTOs blindan nuestro modelo interno y adaptan los datos a las necesidades exactas del cliente.
- ✓ La paginación con Skip y Take es obligatoria para proteger el rendimiento de la API.
- ✓ La inyección de dependencias Scoped mantiene el estado aislado por cada petición HTTP.

GRACIAS

